

Semester Planning Agent

An AI agent that plans a Technion semester like an experienced academic advisor

Team report - everything we built, explained simply
July 2026

The project in 60 seconds

The idea	A student opens a chat website, writes 'I'm starting semester 5, I failed Data Structures, I can't come on Sundays' - and an AI builds them a full semester plan: courses, weekly timetable, exam calendar, risks. If they want changes, they just say so and the SAME plan gets updated.
What makes it special	It is a real AI agent, not a script: the AI decides for itself which checks to run and when the plan is good enough - while our Python code guarantees the hard rules are never broken (an AI is never trusted blindly).
The data	We discovered Technion quietly exposes all course data through a public API (no password needed), and combined it with real student reviews from CheeseFork. Everything is downloaded once, offline - the website never waits on Technion.
Status	Fully working end-to-end on localhost, tested (54 automated checks + live test runs that found and fixed 10 real bugs), total OpenAI spend so far under \$3 of the \$5 budget.

Try it in 30 seconds

1. Start the backend	<code>cd plannerProject && uvicorn main:app --port 8787 --app-dir agent</code>
2. Start the site	<code>python3 -m http.server 4173 --directory site</code>
3. Open	<code>http://127.0.0.1:4173</code> - pick a track, describe your semester, watch it plan live

The OpenAI key lives in agent/.env (gitignored - never commit it).

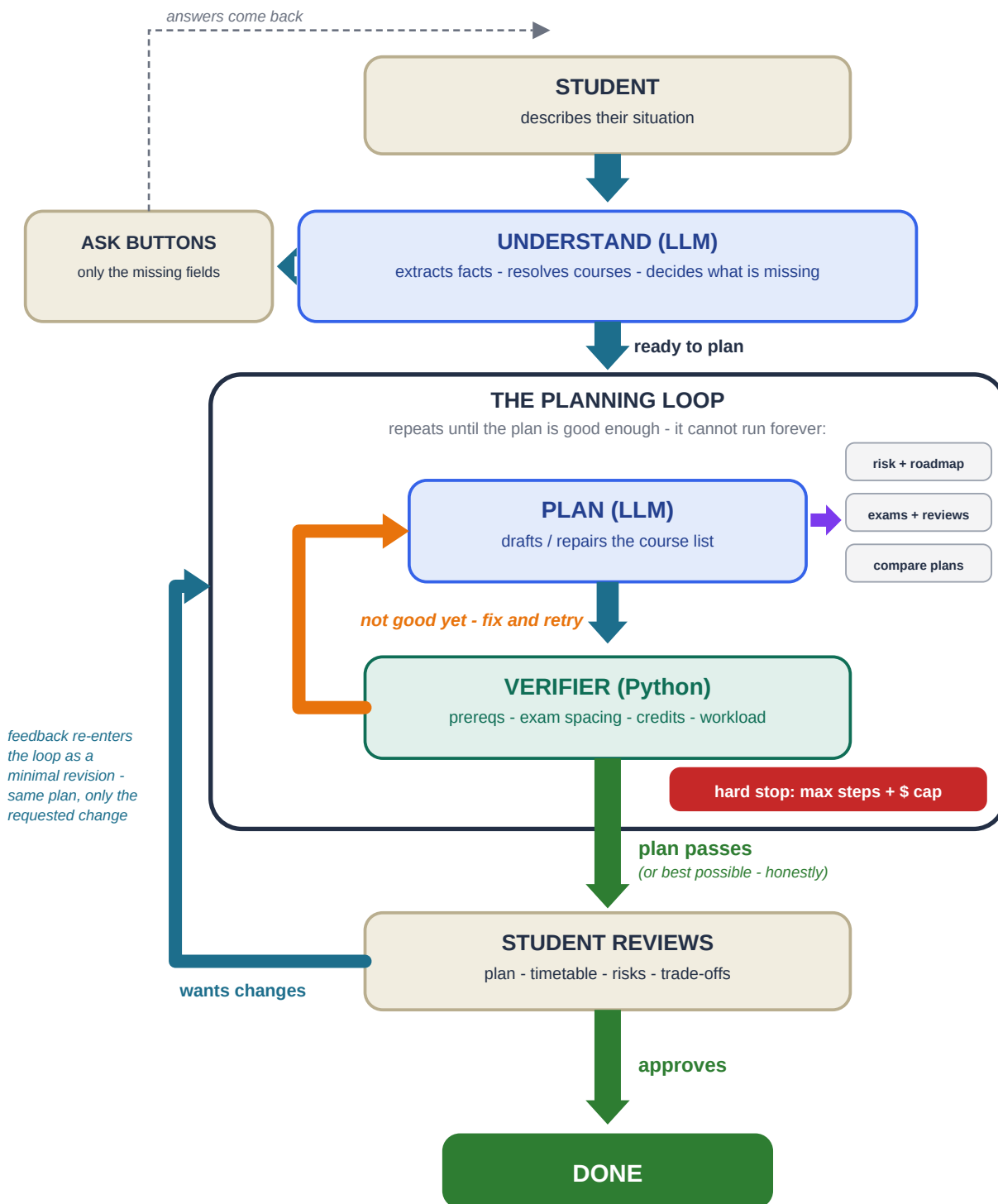
1. The Story - how we got here

The project went through five phases, and each turn taught us something that shaped the final design. Knowing this history explains WHY the code looks the way it does.

Phase	What happened	What we learned
1. Solve the data	Found Technion's public SAP API (course catalog, schedules, exam dates, prerequisites, full degree requirement trees) and CheeseFork's public review database. Built an offline pipeline that packages everything per degree track.	The 'impossible' data problem had a clean solution hiding in plain sight
2. First agent attempt	Gave an LLM free rein with tools and let it chat with the student. It kept asking permission mid-work ('should I proceed?'), asked students for course NUMBERS, and rambled.	Never let the model improvise in front of the user
3. The pipeline version	Rebuilt as a fixed 3-step script: extract info > a Python loop of draft-verify-repair > explain. Reliable and quiet - but honestly a workflow, not an agent: Python made every decision.	Reliability comes from bounded loops + a deterministic verifier
4. The real agent	Rebuilt again: now the MODEL chooses which tools to call and when it is done, inside a bounded loop - all the reliability tricks kept as guardrails around it.	You can have real agent autonomy AND reliability - autonomy just must never touch a student-facing turn
5. Polish + hardening	Live revisions, animations, timetable, Hebrew mode, exports, memory - then a full test campaign that found and fixed 6 real bugs.	Testing IS development: every live test session found something real

2. How it works - the big picture

Read the diagram top to bottom. The two loops are the heart of it: the inner loop keeps improving the plan until it passes verification (with hard limits so it can never run forever), and the outer loop brings the student's feedback back in as a small revision instead of starting over.



Two loops drive the agent: the inner plan-verify loop, and the outer student-feedback loop.
The model is trusted with judgment - never with hard correctness.

One important detail: the model is trusted with judgment (which courses fit this student), but never with hard correctness - after it delivers, Python re-checks everything and fixes violations before the student sees anything.

3. A real conversation, step by step

- **Student:** 'Starting semester 5, failed Data Structures, can't come Sundays, normal pace.'
- **Understand:** one LLM call reads it, turns course names (Hebrew/English/typos) into official course numbers, and decides nothing is missing - straight to planning. If something WERE missing, the site shows buttons for just that field (nobody gets interrogated).
- **Python prepares the ground:** figures out everything a semester-5 student must have already passed (using the prerequisite chains), computes their progress vs a typical student, and builds the shortlist of courses they can ACTUALLY take next semester.
- **The agent plans:** the model drafts a plan around the Data Structures retake (rule #1: retakes come first), checks exam dates and student reviews with its tools, runs the verifier - exam clash found - fixes it, verifies again, checks the plan's risks, and delivers.
- **The student sees** an animated plan: stat tiles, course cards each with WHY it was chosen, a weekly timetable, the exam calendar, the top risk, and an honest note about anything that could not be satisfied.
- **Student:** 'Swap the statistics course, keep everything else.' The agent updates THE SAME plan - one course out, a good replacement in, everything else untouched - and the site shows exactly what changed (kept / removed / added).
- **Student:** 'Is Machine Learning hard?' - answered in seconds from real student reviews, for a tenth of a cent, without touching the plan.

4. The data - where it comes from, what we did to it

Source	What we take	Access
Technion's internal course system (SAP)	Every course: name, credit points, prerequisites, weekly schedule with all class sections, exam dates (moed A+B), which semesters it runs, and the full degree requirement tree per track	Public API, no password
CheeseFork (student community site)	Real student reviews per course: difficulty 1-5, satisfaction 1-5, and written reviews we quote from	Public API, no password

What the pipeline does (runs once, offline)

- Walks the degree requirement tree of a track and collects every course it mentions (137-139 per track)
- Parses the raw prerequisite data into proper AND/OR logic trees
- Precomputes 'what does each course unlock' (the reverse-prerequisite graph) - this powers the retake priority, the risk report, and the graduation roadmap at zero runtime cost
- Attaches the review data to each course
- Writes one JSON file per track; the server loads them at startup - requests never wait on Technion

The tricky data problems we solved

- **No official 'semester 3 course' labels exist** - so we infer a course's typical semester from how deep it sits in the prerequisite chain
- **A mandatory course's prerequisites can live in a different faculty's list** - we follow the real prerequisite chains, not the category labels
- **'The course meets on Sunday' is often true for only ONE of its sections** - we check per-section, which removed half of the false schedule conflicts
- **Course numbers appear with and without leading zeros** - everything is normalized to 8 digits, everywhere

5. The agent, explained simply

'Agent' has a precise meaning: the AI itself decides what to do next - which check to run, whether the plan is good enough, when to stop - the way a person uses a calculator and a checklist. A script follows fixed steps; an agent chooses its steps. Ours chooses (we verified this in live traces: different requests produce different self-chosen tool sequences), inside limits our code enforces.

Its toolbox - 15 tools, all plain Python over our data (no AI inside them)

Group	Tools	Used for
Check the plan	verify_plan, check_invariants	The critic: prerequisites, exam spacing, credits, workload, schedule conflicts, data integrity
Understand the degree	fetch_catalog, query_prereq_graph, assess_progress, roadmap_to_graduation, simulate_future_impact	What is left, what blocks what, is the student behind, what do this semester's choices unlock later
Judge quality	summarize_cheesefork, fetch_exam_dates, risk_report, compare_plans	Are these courses liked, do exams collide, what could go wrong, which of two candidate plans is better
Live with the plan	weekly_schedule_analyzer, exam_study_planner	What the week REALLY looks like (coordinated class sections, overlaps, free days) and whether the exam period leaves real study days - moed A and B
Find + finish	search_courses, deliver_plan	Resolve a course name; hand over the final plan (the model's own 'I am done' decision)

The full tool list (15)

#	Tool	What it does	Called by
1	verify_plan	The critic: prerequisites, offered-next-semester, credit range, section-aware day conflicts, class-time overlaps, exam spacing, workload cap, mandatory floor - pass/fail + itemized issues	model + server
2	check_invariants	Data integrity: already-passed courses, unknown numbers, duplicates - the never-trust-the-model backstop	model + server
3	weekly_schedule_analyzer	Coordinates one class section per course (avoiding excluded days and each other); reports unavoidable overlaps, days on campus, free days, busiest day	model + server
4	exam_study_planner	Chronological exam timeline (moed A + B) with study-days before each exam, crunch flags, and the single swap that most relieves the worst crunch	model
5	risk_report	Stress test: fragile courses (what a repeat failure delays), exam crunch windows, moed B collisions, stacked heavy courses	model + server
6	roadmap_to_graduation	Projects the remaining mandatory requirements across future semesters, assuming the candidate plan completes	model + server
7	simulate_future_impact	Which deferred courses would bottleneck later semesters (gateway scores over the prerequisite graph)	model
8	compare_plans	Objective side-by-side metrics for two candidate plans - the judgment of which wins stays with the model	model
9	summarize_cheesefork	Crowd difficulty (1-5), satisfaction (1-5), review counts and real review excerpts per course	model
10	fetch_exam_dates	Moed A and B exam dates for a set of candidate courses	model
11	search_courses	Resolves a course name (Hebrew / English / partial) to its official number	model
12	deliver_plan	Terminal action: final course list + per-course reasons + the student-facing explanation - the model's own 'I am done' decision	model (terminal)
13	assess_progress	The student vs a typical student at this semester: ahead / on track / behind	pre-injected

#	Tool	What it does	Called by
14	fetch_catalog	Degree requirement audit per category: done, remaining, points left	pre-injected
15	query_prereq_graph	What each failed/outstanding course blocks, directly and transitively, with gateway scores	pre-injected

'Pre-injected' tools run automatically before every turn and their results are handed to the model (saving three paid round-trips); 'model + server' tools are both freely callable by the agent and re-run by the server on the final plan regardless of what the model did.

The safety rules (our code pushes back, once each)

Each rule exists because we watched the model actually make that mistake. A rule rejects the delivery ONCE with a concrete reason; the model then chooses how to fix it. If it fails twice, code enforces the fix.

- You must VERIFY a plan before delivering it
- No half-empty semesters (credits below the floor get sent back)
- No padding the plan with 1-point sport courses (max one)
- If the student complained about a specific problem, the 'fixed' plan may not still contain it
- If the student said 'remove course X', X cannot appear in the delivered plan - period (enforced in code)
- A delivered week can never contain a class-time collision or a course whose prerequisites aren't met - both are stripped in code with an honest note
- The delivery standard: at most 2 open issues, and never 'lazy' ones (low credits, exam clashes) - the agent is sent back until it meets the bar
- Hard limits per turn: max 18 tool calls and a dollar cap - it can never loop forever

The advisor brain (rules from a real Technion advisor interview)

- A failed course that blocks others gets retaken NOW, no matter how hard it is
- The student's pace shapes the courses AROUND the retake - never whether the retake happens
- Exam clustering is the #1 real-world planning mistake - avoid it while choosing, not after
- Target 18 credits (14-18 for a light semester); fill gaps with well-liked electives
- Build order: mandatory skeleton first (retakes + required courses), then like-for-like substitutions, then well-liked electives fill the rest
- When a course can't fit: substitute the nearest-topic elective from the same category, or pull a next-semester mandatory course forward if unlocked
- Day constraints are respected - deliver clash-free, and tell the student exactly what freeing one day would unlock
- If something truly cannot be satisfied - say so honestly and specifically, never hide it

6. Everything the app can do

Talking to it	Natural free text (Hebrew or English) - buttons appear only for genuinely missing info - instant answers to course questions, quoting real reviews - the working card shows the agent's REAL steps live while it thinks
The plan you get	Animated reveal - stat tiles with counting numbers - course cards with a 'why this course' note - weekly timetable with real time slots - exam calendar - top risk + graduation roadmap - honest notes on anything unresolved - confetti when a plan fully passes
Changing it	Say it ('swap X', 'I have a wedding on the 23rd', 'Sundays are fine now', 'actually I passed X') or tap quick-reply chips, or drag a course off the timetable - the SAME plan updates minimally and the diff (kept/removed/added) is shown
Extras	Remembers your session ('continue where you left off?') - export to phone calendar (.ics) - print sheet - shareable PNG card - full Hebrew right-to-left mode - optional soft sounds - two degree tracks supported

7. How we know it works

- **54 automated checks that cost nothing** (run: `bash agent/tests/run_mocked.sh`) - they simulate the model and lock in every guarantee: facts are never forgotten between turns, a revision can never return a stranger plan, 'remove X' is always enforced, guards fire exactly once
- **A live regression suite** - 6 synthetic students run against the real model, checking the hard rules hold
- **A leveled test plan** (TESTING.md) - smoke tests first, then basics, then core features, then hard cases - with a gate between levels
- **Professor-style probes** - a risk-framed request made the model spontaneously run risk analysis and the roadmap before delivering a verified plan; a 'give me two options and choose' request produced five verified candidates and a reasoned pick

Live testing found 10 real bugs - all fixed the same day

What we caught	The fix
A hung network call could freeze a turn for 10 minutes	90-second timeout per call
OpenAI rate-limits crashed turns with an ugly error	Automatic retries with backoff
The budget cap counted the whole conversation, so chats died by turn 3	The cap is now per-turn
'Sundays are fine now' was silently ignored (relaxing a rule was impossible)	The state merge now distinguishes 'not mentioned' from 'explicitly cleared'
A totally unconstrained student could still get a failing plan, silently	New guard: fixable issues send the plan back once
A 13-credit plan slipped past a 14-credit minimum	The floor is now strict
Delivered weeks contained overlapping class times (never checked anywhere)	Coordinated section chooser + hard strip: a clashing week is never delivered
A plan shipped with 5 open issues and no guard reacted	Issue-budget guard: max 2 issues, no fixable ones, pushes back twice
The model knowingly included a prerequisite-locked course as an 'anchor'	Locked courses are stripped in code on every delivery path
Moed B exams 0-1 days apart were invisible	<code>exam_study_planner</code> covers moed A+B; crunches appear in the risk report

8. Team handbook

Where everything lives

Path	What it is
pipeline/	Offline data fetching from Technion + CheeseFork (run once per track)
data/track_*.json	The packaged course bundles the server loads
agent/agent_react_loop.py	THE AGENT - the bounded tool-choosing loop and all its guards
agent/tools.py	All 13 tools (pure Python, no AI inside)
agent/agent_loop.py	Understanding/extraction + shared helpers + the old pipeline (kept as fallback)
agent/main.py	The web server: /chat, /chat/stream (live steps), /tracks, /health
agent/tests/	The automated checks + live scenario suite
site/index.html	The whole website - one file, no build step
TESTING.md	The leveled test plan with exact prompts to try
agent/.env	The OpenAI key + settings (gitignored - never commit)

Mini glossary

Agent	An AI that chooses its own next step (which tool, when to stop) instead of following a fixed script
Tool	A plain Python function the AI can call - e.g. 'verify this plan' - always deterministic, never another AI
Verifier	Our Python critic that checks a plan against every hard rule and lists exactly what is wrong
Guard	A one-time rejection our code gives a bad delivery, with the reason - the AI decides how to fix it
known_context	The memory packet the site sends back every turn: settled facts + the current plan, so nothing is ever re-asked
Track bundle	One JSON file with everything about a degree track: courses, schedules, exams, reviews, prerequisite graph

Honest limitations (so nobody is surprised)

- Runs on localhost only so far - public deployment is the next milestone
- The intake buttons cannot yet express 'I have not reached that course yet' (only passed/failed)
- Calendar export approximates the semester's start; exams are exact, weekly classes may need a nudge
- The same question can produce different (equally valid) plans - that is how LLMs are